



Name: Gargi Sukhatankar  
Roll No: B027

SAP ID: 60019220125  
Branch: CSE(ICB)

## EXPERIMENT 5

**Aim:** Study of Neo 4j Commands.

**Objective:**

- To understand the fundamentals of Neo4j as a graph database.
- To learn how to create, query, and manipulate graph data using Cypher query language.
- To explore practical applications of graph databases.

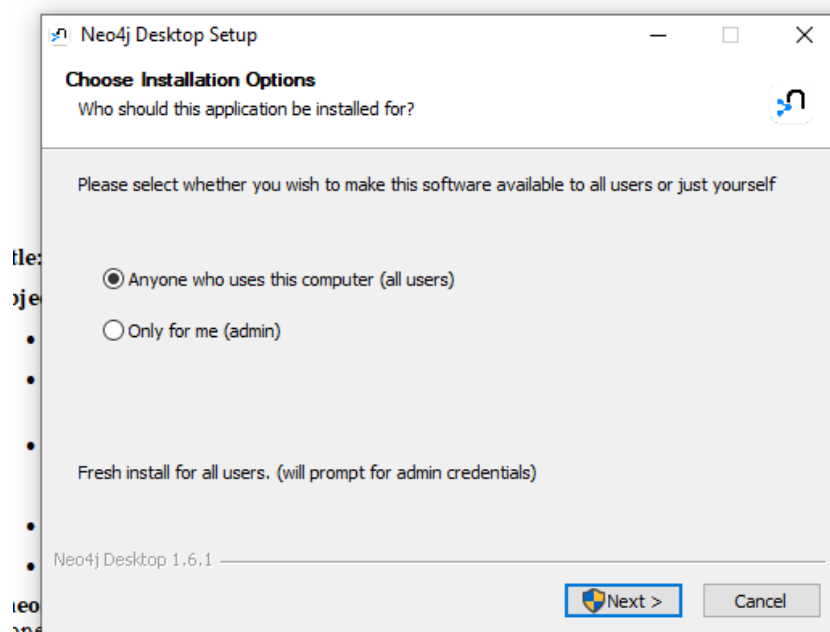
**Materials Required:**

- Computer with Neo4j Desktop or Neo4j Aura (cloud version) installed.
- Internet connection.

**Theory:** Neo4j is a graph database that uses nodes, relationships, and properties to represent and store data. It follows the property graph model, which is ideal for highly interconnected data. Cypher is the query language used in Neo4j, designed to handle pattern matching with ease.

**Screenshots:**

Step 1: Installation and Setup



ich is ideal for highly interconnected data. Cypher is the query language

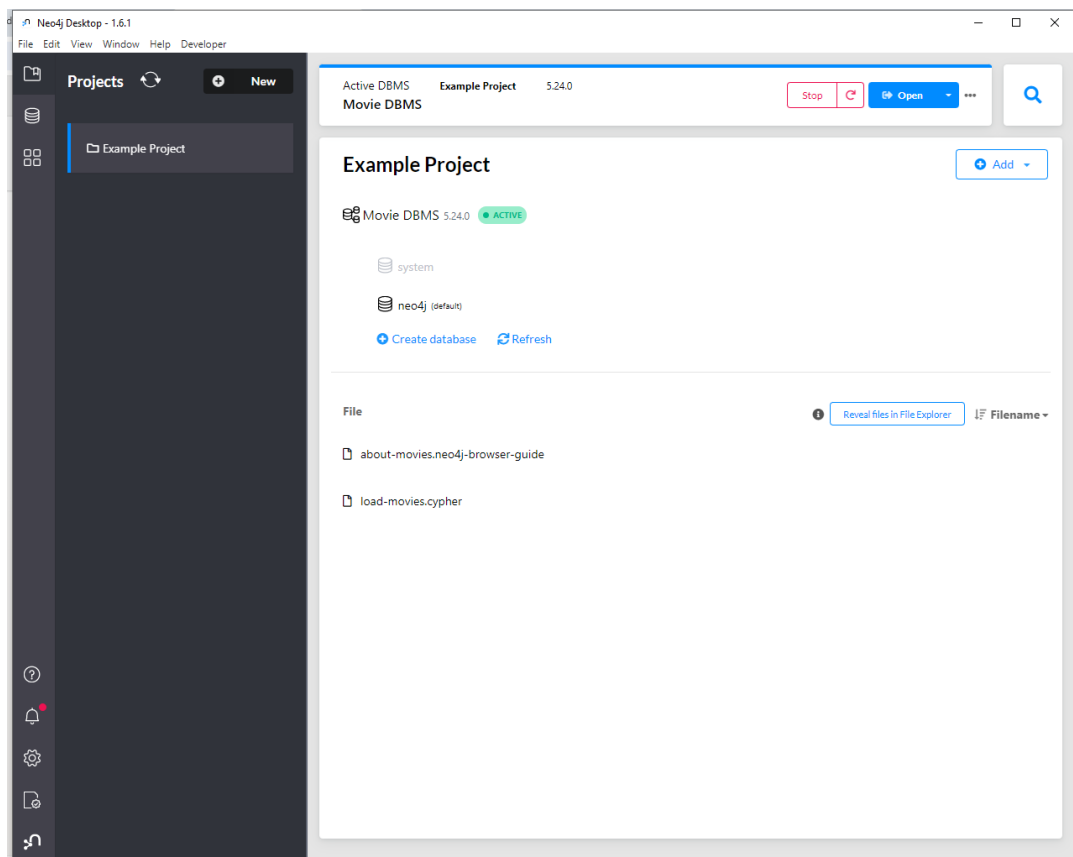
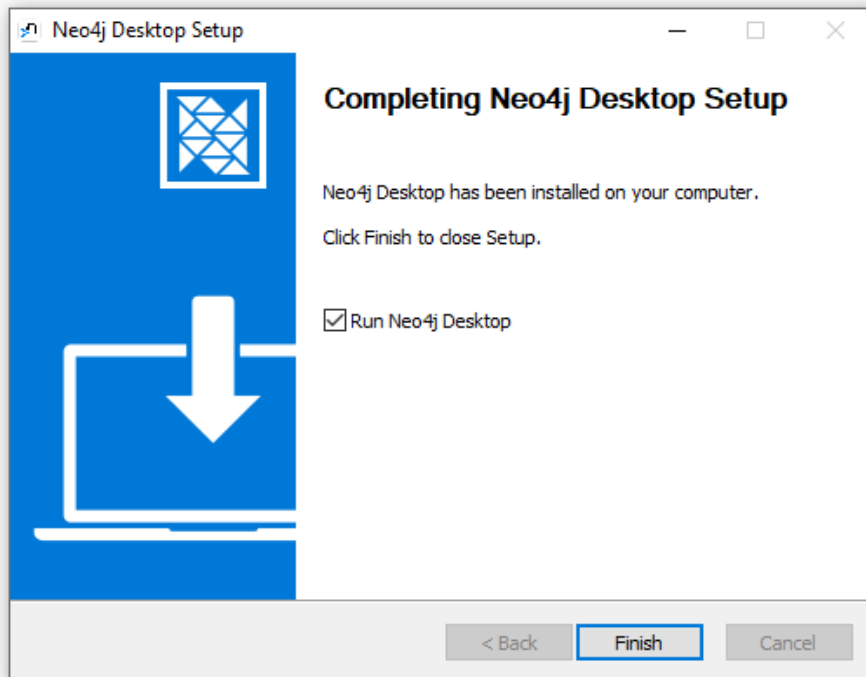


SHRI VILEPARLE KELAVANI MANDAL'S  
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING  
(Autonomous College Affiliated to the University of Mumbai)  
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar  
Roll No: B027

SAP ID: 60019220125  
Branch: CSE(ICB)





Name: Gargi Sukhatankar  
Roll No: B027

SAP ID: 60019220125  
Branch: CSE(ICB)

Step 2: Create Graph Data Use the following Cypher queries to create a sample social network graph.

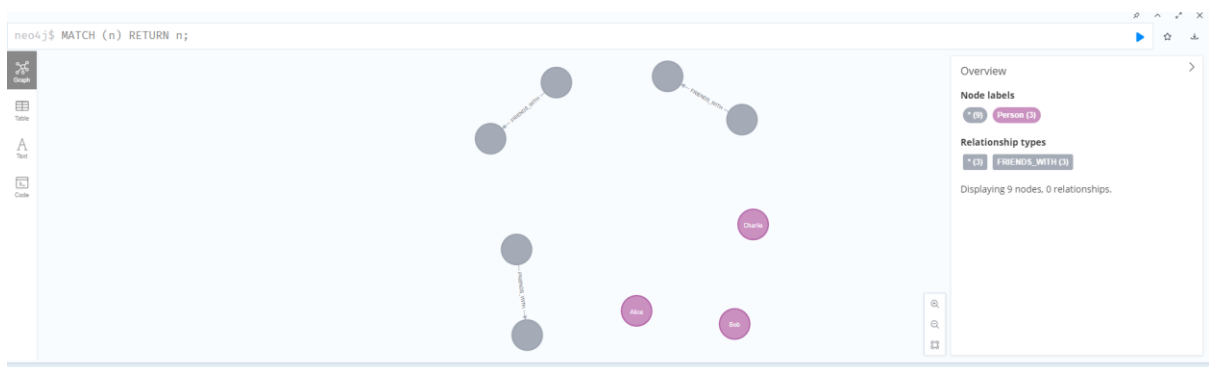
```
// Create nodes
CREATE (Alice:Person {name: 'Alice', age: 25})
CREATE (Bob:Person {name: 'Bob', age: 30})
CREATE (Charlie:Person {name: 'Charlie', age: 35})

// Create relationships
CREATE (Alice)-[:FRIENDS_WITH]->(Bob)
CREATE (Bob)-[:FRIENDS_WITH]->(Charlie)
CREATE (Alice)-[:FRIENDS_WITH]->(Charlie)
```

Step 3: Query the Graph

- Retrieve all nodes:

```
MATCH (n) RETURN n;
```



- Find friends of Alice:

```
MATCH (alice:Person {name: 'Alice'})-[:FRIENDS_WITH]->(friend) RETURN friend;
```



Name: Gargi Sukhatankar

SAP ID: 60019220125

Roll No: B027

Branch: CSE(ICB)

```
neo4j$ MATCH (Alice:Person {name: 'Alice'})-[:FRIENDS_WITH]->(friend) RETURN friend;
```

Table

(no changes, no records)

Code

Completed after 23 ms.

- Count relationships:

```
MATCH ()-[r]->() RETURN type(r), count(r);
```

```
neo4j$ MATCH ()-[r]->() RETURN type(r), count(r);
```

|   | type(r)        | count(r) |
|---|----------------|----------|
| 1 | "FRIENDS_WITH" | 4        |

Started streaming 1 records after 46 ms and completed after 48 ms.

#### Step 4: Modify the Graph

- Add a new person and connect them to existing nodes:

```
CREATE (David:Person {name: 'David', age: 28})  
CREATE (David)-[:FRIENDS_WITH]->(Alice);
```



Name: Gargi Sukhatankar  
Roll No: B027

SAP ID: 60019220125  
Branch: CSE(ICB)

```
neo4j$ CREATE (David)-[:FRIENDS_WITH]→(Alice);
```

Table

Code

Created 2 nodes, created 1 relationship, completed after 4 ms.

Created 2 nodes, created 1 relationship, completed after 4 ms.

- Update node property

```
MATCH (p:Person {name: 'Alice'}) SET p.age = 26;
```

```
neo4j$ MATCH (p:Person {name: 'Alice'}) SET p.age=26;
```

Table

Code

Set 1 property, completed after 11 ms.

Set 1 property, completed after 11 ms.

#### Step 5: Analyse Graph Patterns

- Find mutual friends:



Name: Gargi Sukhatankar

SAP ID: 60019220125

Roll No: B027

Branch: CSE(ICB)

```
MATCH (a:Person)-[:FRIENDS_WITH]->(m:Person)<-[:FRIENDS_WITH]-(b:Person)
WHERE a.name = 'Alice' AND b.name = 'Bob'
RETURN m;
```

```
neo4j$ MATCH (a:Person)-[:FRIENDS_WITH]->(m:Person)<-[:FRIENDS_WITH]-(b:Person) WHERE a.name = 'Alice' AND b.name = 'Bob' RETURN m;
```

(no changes, no records)



Completed after 15 ms.

### Step 6: Updating Data

```
MATCH (n: Person {name: 'David'}) SET n.age=20;
```

```
neo4j$ MATCH (n: Person {name: 'David'}) SET n.age=20;
```



Table



Code

Set 1 property, completed after 7 ms.

### Step 7: Deleting Data

- Delete a relationship:

```
MATCH (a:Person)-[r:KNOWS]->(b:Person) DELETE r;
```



Name: Gargi Sukhatankar

SAP ID: 60019220125

Roll No: B027

Branch: CSE(ICB)

```
neo4j$ MATCH (a:Person)-[r:KNOWS]→(b:Person) DELETE r;
```



Table

(no changes, no records)



Code

Completed after 1 ms.

- Delete a node (and its relationships):

```
neo4j$ MATCH (n: Person {name: 'David'}) DETACH DELETE n;
```

```
neo4j$ MATCH (n: Person {name: 'David'}) DETACH DELETE n;
```



Table

Deleted 1 node, completed after 57 ms.



Code

Deleted 1 node, completed after 57 ms.

Step 8: Advanced Queries



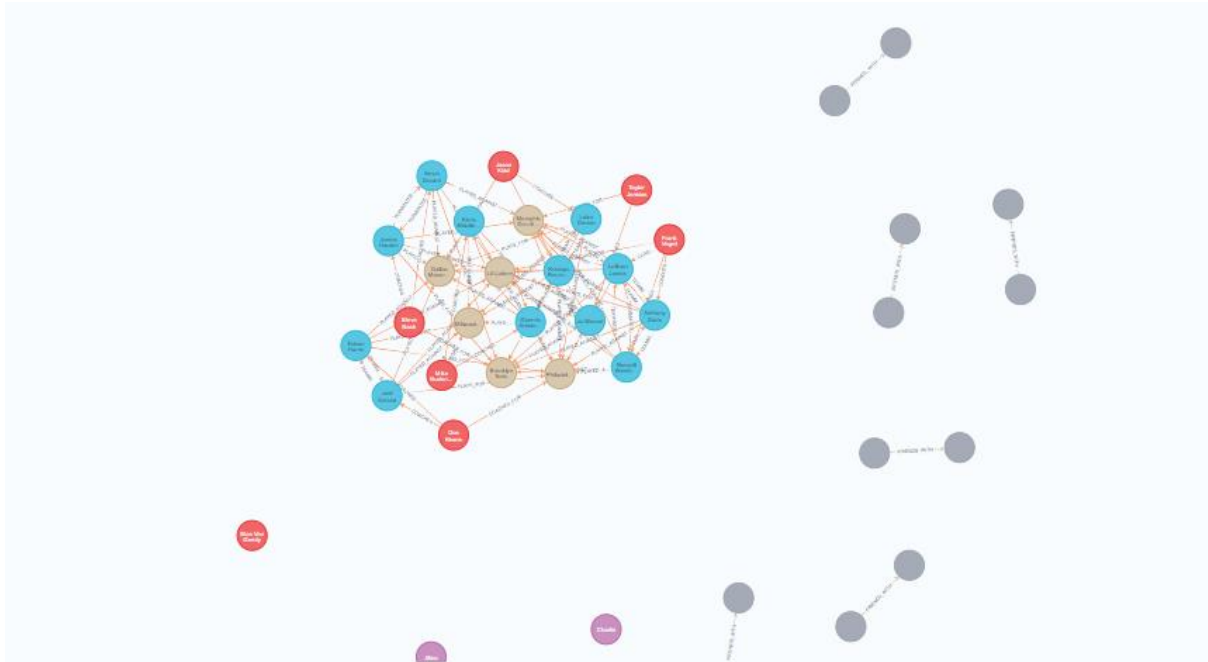
SHRI VILEPARLE KELAVANI MANDAL'S  
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING  
(Autonomous College Affiliated to the University of Mumbai)  
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



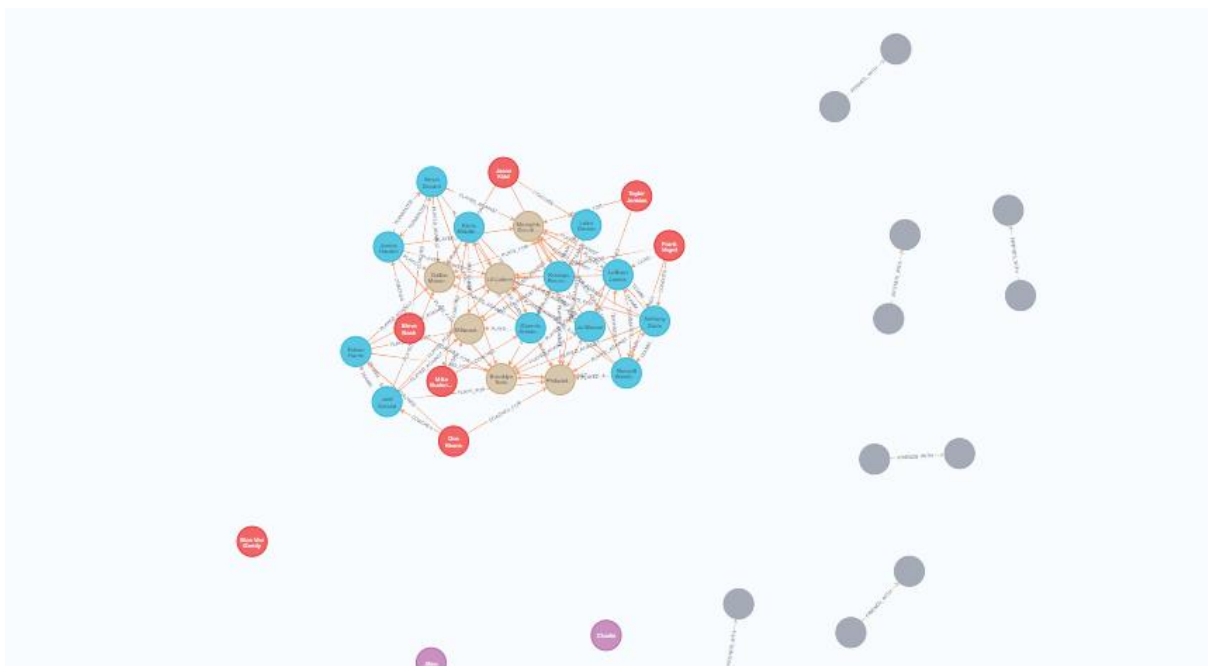
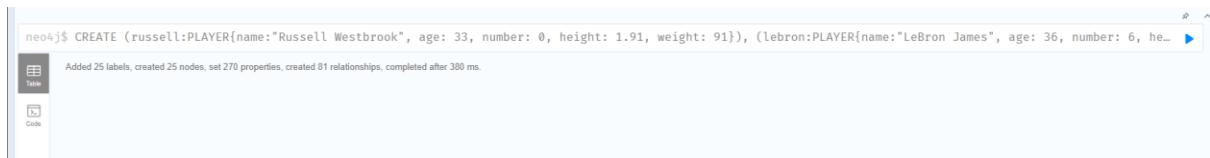
Name: Gargi Sukhatankar  
Roll No: B027

SAP ID: 60019220125  
Branch: CSE(ICB)

```
MATCH p=shortestPath((a:Person {name: 'Alice'})-[:FRIENDS_WITH*]-(b:Person {name: 'Charlie'})) RETURN p;
```



Github Database:





SHRI VILEPARLE KELAVANI MANDAL'S  
**DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING**  
(Autonomous College Affiliated to the University of Mumbai)  
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar  
Roll No: B027

SAP ID: 60019220125  
Branch: CSE(ICB)

**Conclusion:**

Neo4j provides an intuitive way to manage connected data using graph structures. The experiment demonstrated how to create, query, and manipulate graph data efficiently.